

SOFTWARE MODELLING

AMIRA AJANAKU (25111699) / SENZO LUKHELE (24691497) / JAY MACASKILL (25198387)

08 SEPTEMBER 2026 / COS214 PRACTICAL 4 (TASK FORGE)

Task 1

Task Forge is a work management system for software development projects, modelling work breakdown structures where Projects, Tasks and Epics are treated uniformly and move through a lifecycle. Projects contain Epics (or subgroups) and Epics contain Tasks. Tasks can also exist independently. The system allows for flexible traversal (e.g. view all items or only active ones) of the work hierarchy and addition of responsibilities such as priority and logging.

We have elected to do a software delivery hierarchy. This system acts as a means to track work being done and the lifecycle of Tasks/Projects/Epics.

- The Composite pattern is used such that a WorkGroup and a WorkItem can be treated uniformly through a common WorkComponent interface.
- A WorkItem could represent something such as a Task whilst a WorkGroup could represent a Project or an Epic, etc. However, all WorkComponents have a lifecycle which ranges from Todo → InProgress → Blocked → Done. This serves as the State design pattern within the system.
 - State-dependent behaviour is given to the TaskState object which allows the item's behaviour to change without conditionals, rather based on the TaskState object's internal workings.
 - Using the State pattern avoids large logical statements
 - New states can be added without modifying WorkItem
 - Following the Open/Closed principle
 - State transitions are handled within state objects
 - States are owned by the WorkItem and destroyed when the WorkItem is destroyed.
- Further, we make use of a TaskDecorator to decorate a task further by priority with the PriorityDecorator, or we can make use of a LoggingDecorator as well – this serves as the Decorator design pattern in the system.
 - Decorators forward calls to the WorkComponent they decorate and decorators can be stacked.
 - The outermost decorator owns all wrapped components, ownership flows from outer.
- The Iterator design pattern allows us to traverse through a Composite. There is a DepthFirstIterator which will iterate through the whole Composite tree as well as an ActiveOnlyIterator which is filtered to traverse through only active tasks.
 - Further we note the use of “snapshot iterators” both iterators must use a snapshot approach. The tree is flattened at construction time, so structural changes during

iteration are not reflected which provides safe traversal without concurrent modification issues – this is the traversal modification policy

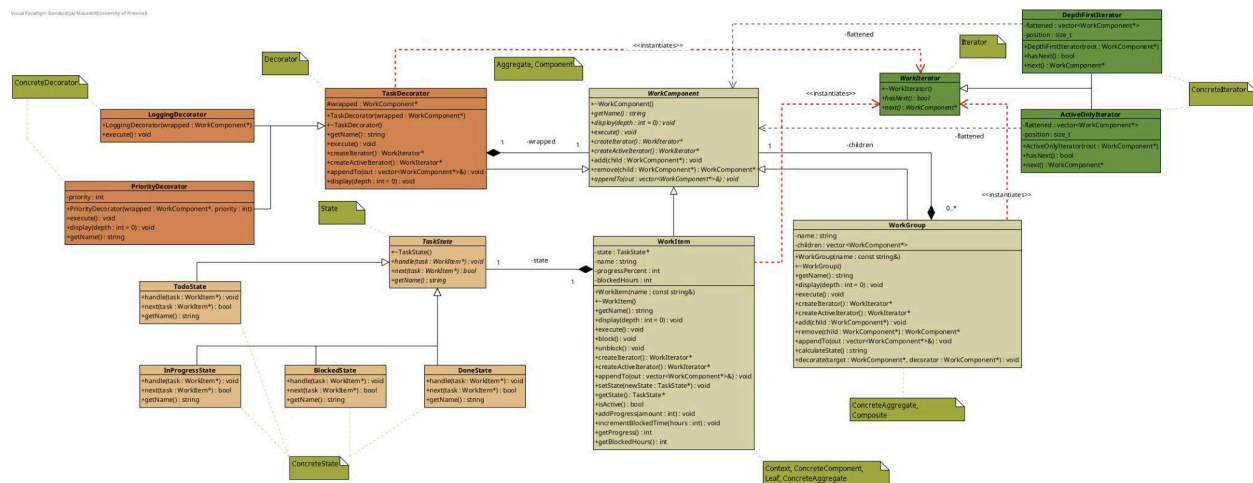
- This prevents concurrent modification issues
- Allows safe traversal even through structure changes
- Does not require invalidation logic
- Each WorkComponent implements createIterators, which allows obtaining of iterators without knowing if the component is a leaf or composite.
- These patterns interact meaningfully to model a realistic system. Tasks/Projects/Epics do not exist in the real world independent of a lifecycle, priority level, logs or a means to go through all Tasks/Projects/Epics and a means to filter which of these are active and need tending to.
 - The patterns therefore do not act independently, rather they interleave and assist with increasing the meaningfulness of the design.

Notably on memory management:

- WorkGroup owns its children as a vector of pointers and the destructor deletes the children. If a WorkComponent is removed from a WorkGroup, the removed WorkComponent is returned to the caller. This allows reallocation of items from within groups to single items or reallocation of groups from within a group to an independent group.
 - This creates a clear destruction policy
 - The caller owns the root which creates explicit lifecycle management, deletion of the root cascades to all children
- Decorator owns its wrapped component. The outermost decorator's destructor deletes what it wraps, which cascades through nested decorators all the way to the underlying WorkItem. A WorkComponent has one owner at any time (either a WorkGroup if plain and undecorated) or the outermost decorator.
 - To decorate an item already in a WorkGroup, ownership is transferred using decorate() which swaps the group's pointer to the child from the item itself to the decorator, meaning the WorkGroup doesn't delete an item a decorator also owns.

Pattern	Participants	
Iterator	Iterator	WorkIterator
	ConcreteIterator	DepthFirstIterator, ActiveOnlyIterator
	Aggregate	WorkComponent
	ConcreteAggregate	WorkGroup, WorkItem
Composite	Component	WorkComponent
	Leaf	WorkItem

	Composite	WorkGroup
Decorator	Component	WorkComponent
	ConcreteComponent	WorkItem
	Decorator	TaskDecorator
	ConcreteDecorator	PriorityDecorator, LoggingDecorator
State	State	TaskState
	ConcreteState	TodoState, InProgressState, BlockedState, DoneState
	Context	WorkItem



Tasks 2 & 3

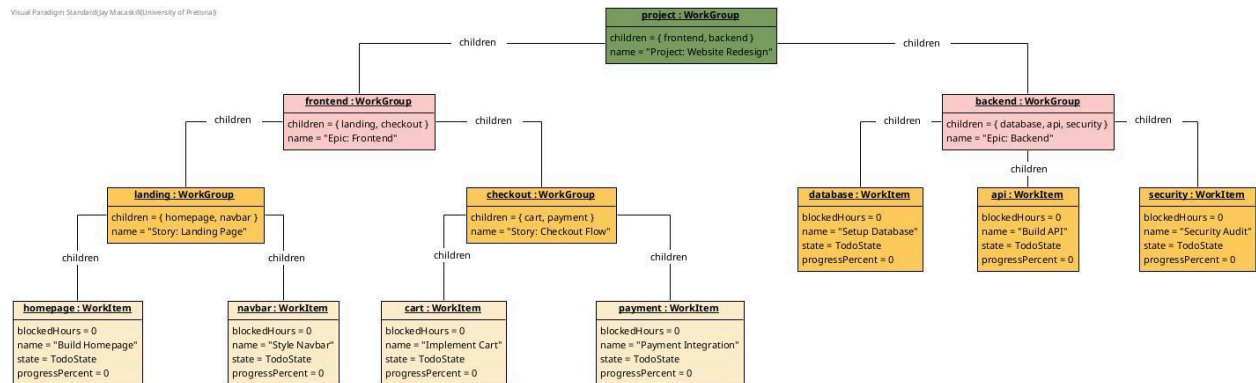
The main.cpp demonstrates the following:

- Traversal of nested objects
 - DepthFirstIterator recursively flattens the tree via `appendTo()`, visiting WorkGroups and WorkItems at every level from the root down to the deepest task
- Lifecycle/state-dependent behaviour
 - A WorkItem changes behaviour based on its current TaskState, with `execute()` delegating to state-specific `handle()` methods that add progress, log blocked time, or reject actions
- At least one decorated object participating in normal system behaviour

- The homepage decorator chain (PriorityDecorator wrapping LoggingDecorator) is added to the hierarchy and executed as part of the normal project workflow, logging and prioritising the task
- At least one runtime change to the structure, state, decoration, or a combination of these
 - main.cpp moves the Build API task from the Backend epic to the Frontend epic during execution using remove() and add(), demonstrating a structural runtime change
- A genuine recursive part-whole hierarchy with meaningful leaf and grouping behaviour
 - The hierarchy Project → Epic → Story → Task uses WorkGroup as composite containers and WorkItem as leaves, with display() recursively printing the entire tree
- Traversal without exposing the hierarchy's internal representation to client code
 - Client code calls createIterator() on a WorkComponent and uses hasNext()/next(), with the internal children vector remaining private and only accessed via appendTo()
- At least two meaningful traversal behaviours over the same hierarchy
 - DepthFirstIterator visits every component in the tree, while ActiveOnlyIterator filters to only WorkItems where isActive() returns true
- Two independent traversals over the same runtime structure
 - main.cpp creates both a DepthFirstIterator and an ActiveOnlyIterator on the same project root and uses them independently without interference
- State-dependent behaviour with valid and invalid transitions
 - A task moves Todo → InProgress → Done via valid transitions, while Done → next() is rejected as an invalid terminal-state transition
- At least two meaningful decorators that can be applied at runtime, including a case where decorators are stacked
 - PriorityDecorator boosts progress or unblocks tasks, while LoggingDecorator timestamps execution; they are stacked as PriorityDecorator(LoggingDecorator(task)) in main.cpp
- Sensible ownership and clean polymorphic destruction
 - WorkGroup owns its children and deletes them in its destructor; TaskDecorator owns its wrapped component and deletes it, ensuring cascading cleanup with no memory leaks
- Iterators encapsulate traversal through WorkIterator with hasNext()/next() whilst STL vectors are used internally for flattening and are not exposed directly to any client code

Task 4

Visual Paradigm Standard (Jay M. Karkkainen/University of Pretoria)



UML Object Diagram Showing The Following Hierarchy (main.cpp)

```

cout << "\n [COMPOSITE] BUILDING THE HIERARCHY \n";

// Project

WorkGroup* project = new WorkGroup("Project: Website Redesign");

// Epics

WorkGroup* frontend = new WorkGroup("Epic: Frontend");

WorkGroup* backend = new WorkGroup("Epic: Backend");

// Stories

WorkGroup* landing = new WorkGroup("Story: Landing Page");

WorkGroup* checkout = new WorkGroup("Story: Checkout Flow");

// Tasks

WorkItem* homepage = new WorkItem("Build Homepage");

WorkItem* navbar = new WorkItem("Style Navbar");

WorkItem* cart = new WorkItem("Implement Cart");

WorkItem* payment = new WorkItem("Payment Integration");

WorkItem* database = new WorkItem("Setup Database");
  
```

```
WorkItem* api = new WorkItem("Build API");

WorkItem* security = new WorkItem("Security Audit");


cout << "\n🌳 BUILDING THE TREE 🌳\n";

landing->add(homepage);

landing->add(navbar);


checkout->add(cart);

checkout->add(payment);


frontend->add(landing);

frontend->add(checkout);


backend->add(database);

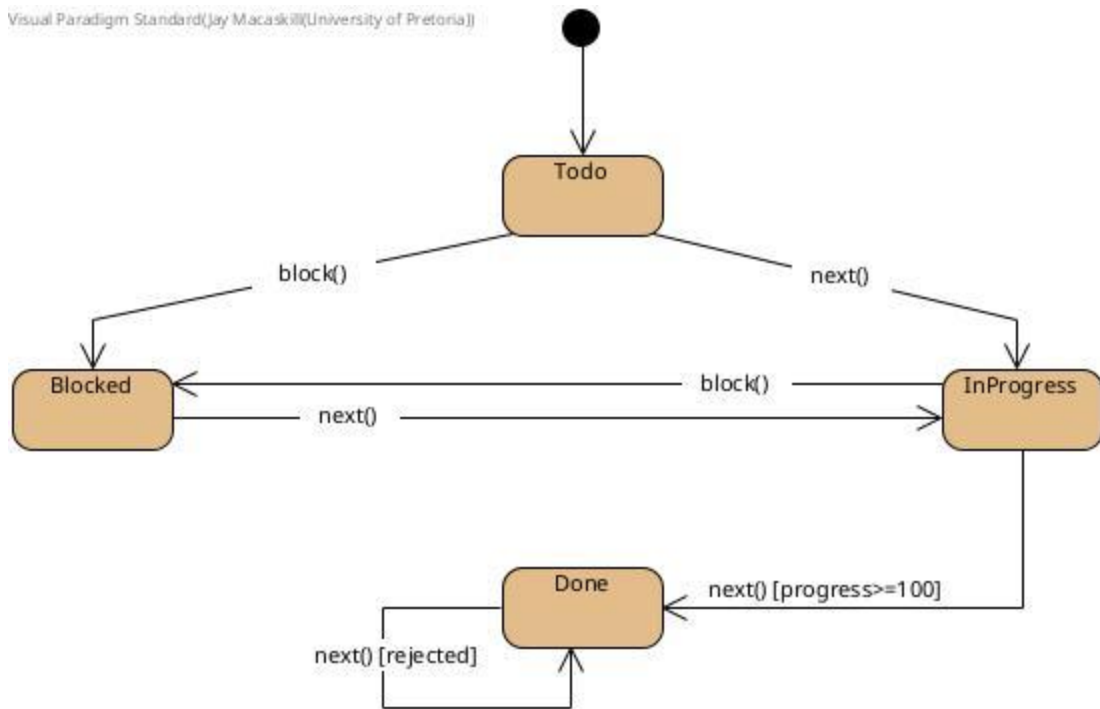
backend->add(api);

backend->add(security);

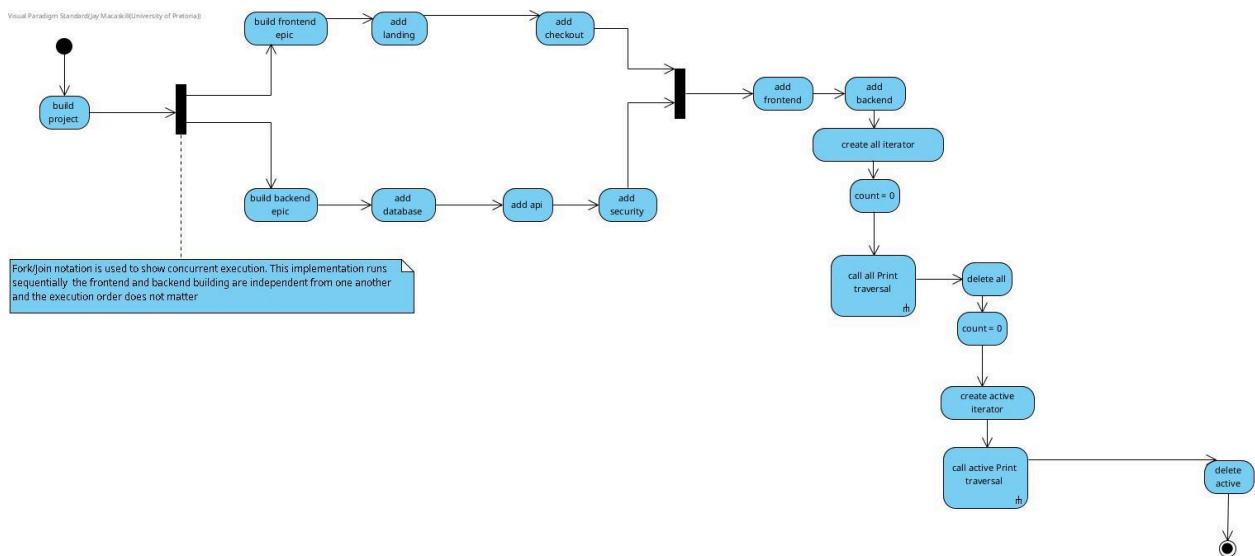

project->add(frontend);

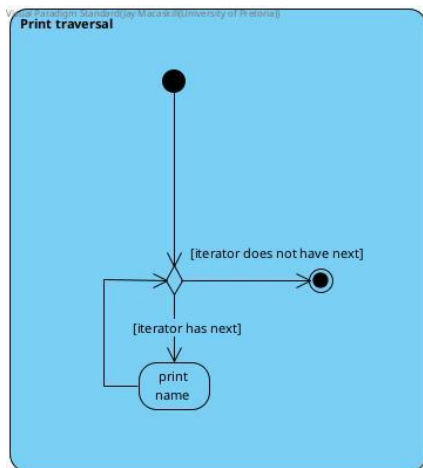
project->add(backend);

cout << "\n✅ Project built successfully!\n";
```



UML State Diagram Showing the Lifecycle of a WorkItem





UML Activity diagram for Building and Traversing the Hierarchy based on the code below from main.cpp

```

cout << "\n📁 [COMPOSITE] BUILDING THE HIERARCHY 📁\n";

// Project
WorkGroup* project = new WorkGroup("Project: Website Redesign");

// Epics
WorkGroup* frontend = new WorkGroup("Epic: Frontend");
WorkGroup* backend = new WorkGroup("Epic: Backend");

// Stories
WorkGroup* landing = new WorkGroup("Story: Landing Page");
WorkGroup* checkout = new WorkGroup("Story: Checkout Flow");

// Tasks
WorkItem* homepage = new WorkItem("Build Homepage");
WorkItem* navbar = new WorkItem("Style Navbar");
WorkItem* cart = new WorkItem("Implement Cart");
WorkItem* payment = new WorkItem("Payment Integration");
WorkItem* database = new WorkItem("Setup Database");
  
```

```
WorkItem* api = new WorkItem("Build API");  
  
WorkItem* security = new WorkItem("Security Audit");
```

```
cout << "\n🌳 BUILDING THE TREE 🌳\n";
```

```
landing->add(homepage);
```

```
landing->add(navbar);
```

```
checkout->add(cart);
```

```
checkout->add(payment);
```

```
frontend->add(landing);
```

```
frontend->add(checkout);
```

```
backend->add(database);
```

```
backend->add(api);
```

```
backend->add(security);
```

```
project->add(frontend);
```

```
project->add(backend);
```

```
cout << "\n✅ Project built successfully!\n";
```

```
project->display();
```

```
// === This segment works well for an activity diagram on traversal ===  
//
```

```
cout << "\n📋 [ITERATOR] TWO INDEPENDENT TRAVERSALS 📋\n";
```

```
cout << "\n📋 ALL TASKS 📋\n";
```

```

WorkIterator* all = project->createIterator();

int count = 0;

while (all->hasNext())
{
    cout << "    " << ++ count << ". " << all->next()->getName() <<
endl;

    delete all;

    cout << "    ➡ " << count << " components visited\n";

    count = 0;

    cout << "\nStarting work on the database...\n";

    database->execute();

    database->execute();

    cout << "\n📋 ACTIVE TASKS 📋\n";

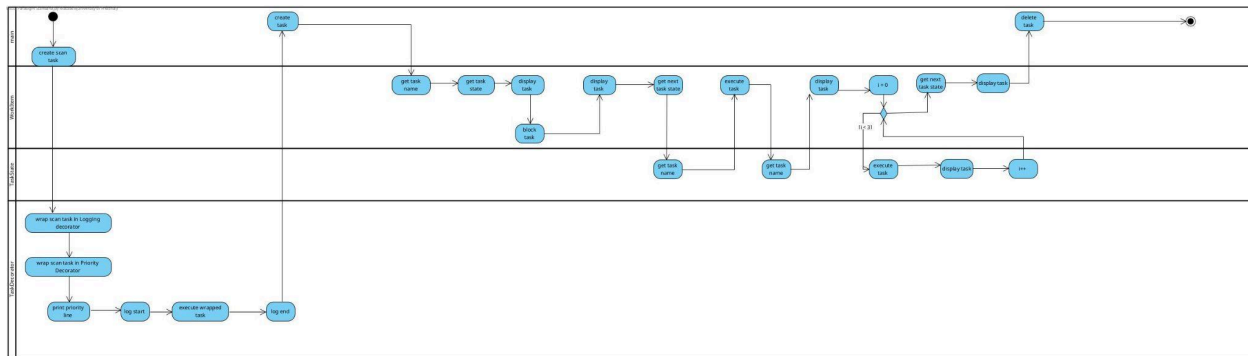
    WorkIterator* active = project->createActiveIterator();

    while (active->hasNext())
    {
        cout << "    " << ++ count << ". " << active->next()->getName() <<
endl;

        delete active;

        cout << "    ➡ " << count << " active components visited\n";

```



UML Activity Diagram showing the Decorator and State Lifecycle, with swimlanes, based on the following code from main.cpp

```

    cout << "\n🎀 [DECORATOR] STACKED DECORATORS AT RUNTIME 🎀\n";

    WorkItem* scan = new WorkItem("Security Scan");

    WorkComponent* decorated = new PriorityDecorator(new
LoggingDecorator(scan), 1);

    cout << "    Decorated task " << decorated->getName() << " with Logging
and Priority decorators. Executing decorated task...\n";

    decorated->execute();

    delete decorated; // Decorator owns the task it wraps

    // === This segment works well for a state diagram
    // and for an activity diagram with state/configuration workflow === //

    cout << "\n🎯 [STATE] TASK LIFECYCLE 🎯\n";

    WorkItem* task = new WorkItem("Crunching Numbers");

    // Initial state: Todo

    cout << "    Task: " << task->getName() << "\n";

    cout << "        Initial: " << task->getState()->getName() << "\n";

    cout << "        Display: ";

        task->display();

    cout << "\n";

```

```

// Todo → Blocked

task->block();

task->display();

task->getState()->next(task);

// Todo → InProgress

task->getState()->next(task);

cout << "          After next(): " << task->getState()->getName() << "\n";

task->execute(); // adds progress

cout << "          After execute: " << task->getState()->getName();

task->display();

// InProgress → Blocked

task->block();

task->display();

// Blocked → InProgress

task->getState()->next(task);

// Once progress >= 100, InProgress → Done

for (int i = 0; i < 3; i++) { task->execute(); task->display(); }

// Invalid transition, Done state is terminal

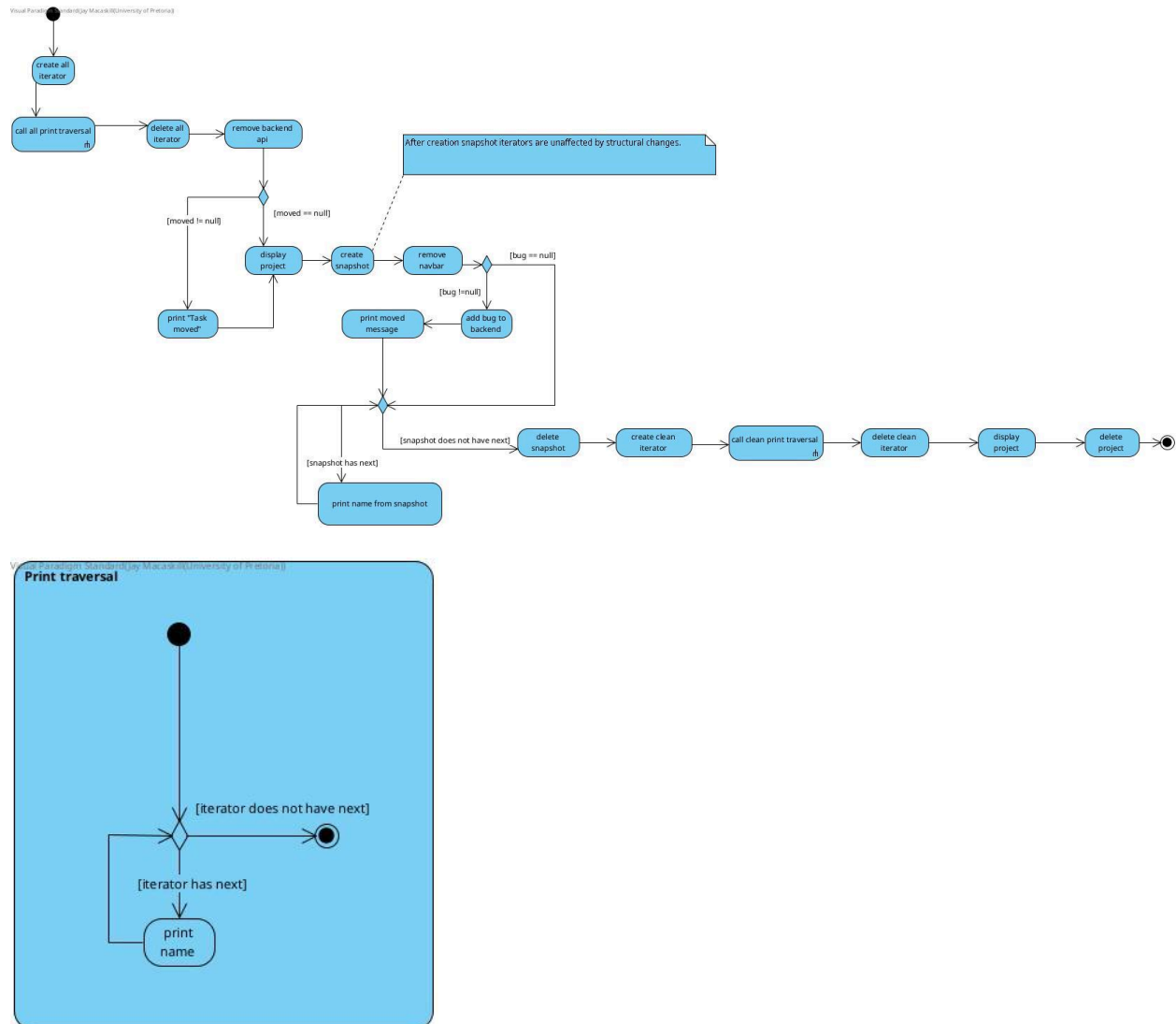
cout << "          Invalid transition: Done → Next\n";

task->getState()->next(task);

task->display();

```

```
delete task;
```



UML Activity Diagram showing the Iterator Policy, based on the following code from main.cpp

```

    cout << "\n [RUNTIME CHANGE] MOVING A TASK BETWEEN GROUPS \n";
    cout << "    Before moving 'Build API':\n";
    cout << "        Backend has: ";
    WorkIterator* before = project->createIterator();
    while (before->hasNext())
    {
        cout << before->next()->getName() << " ";
    } delete before;
  
```

```

cout << "\n";

cout << "\n    🔄 Moving 'Build API' from Backend to Frontend...\n";
WorkComponent* moved = backend->remove(api);
if (moved)
{
    frontend->add(moved);
    cout << "    ✅ Task moved!\n";
}

cout << "\n    Hierarchy after moving:\n";
project->display();

cout << "\n🔥 [TRAVERSAL] ITERATORS HANDLING RUNTIME CHANGES 🔥\n";
cout << "    Building snapshot iterator before moving...\n";

WorkIterator* snapshot = project->createActiveIterator();
WorkComponent* bug = landing->remove(navbar);
if (bug)
{
    backend->add(bug);
    cout << "    ✅ " << bug->getName() << " moved from Frontend to
Backend\n";
}

cout << "Old snapshot showing old structure:\n";
while (snapshot->hasNext())
{
    cout << "    - " << snapshot->next()->getName() << endl;
}

```

```

    } delete snapshot;

    cout << "\nEach time you use an iterator, delete it 🗑️\n";

    cout << "Each time you need an iterator, make a new one ✨" << endl <<
"\n";

    WorkIterator* clean = project->createActiveIterator();
    while (clean->hasNext())
    {
        cout << "    - " << clean->next()->getName() << endl;
    } delete clean;

    cout << "\n📁 FINAL PROJECT HIERARCHY 📁\n";

    project->display();

```

Task 5

Note that we have both a static main and a dynamic demo.cpp. The main.cpp is used for fitchfork and static tests and ensuring good coverage whilst demo.cpp may not be submitted to fitchfork as it requires input. The seg fault below was found whilst testing the standalone interactive demo.cpp.

While running the interactive demo from demo.cpp, selecting option 7 (“Everything At Once”) caused the program to crash with a seg fault shortly after the “Composite Removal” section printed. To investigate, we made use of GDB with a breakpoint set on removingComposite()

```

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./demo...
(gdb) break removingComposite
Breakpoint 1 at 0x498f: file demo.cpp, line 181.
(gdb) run
Starting program: /home/finn/Desktop/code-library/practicals/practical_4/demo

```

Continuing from the breakpoint, the program printed its normal Composite Removal output, including the warning “⚠️ This WorkComponent could not be found in this WorkGroup!” before crashing with SIGSEV inside WorkGroup::appendTo

```
Program received signal SIGSEGV, Segmentation fault.
0x000055555555d916 in WorkGroup::appendTo (this=0x5555555788e0,
    out=std::vector of length 7, capacity 8 = {...}) at WorkGroup.cpp:166
166             children[i]->appendTo(out);
```

A backtrace (bt) confirmed the call chain that led to the crash

```
(gdb) bt
#0  0x000055555555d916 in WorkGroup::appendTo (this=0x5555555788e0,
    out=std::vector of length 7, capacity 8 = {...}) at WorkGroup.cpp:166
#1  0x00005555555558c22 in status () at demo.cpp:202
#2  0x00005555555559431 in main () at demo.cpp:296
```

Inspecting the children vector of the failing WorkGroup instance at the crash revealed the problem (one element was a null pointer)

```
(gdb) print children
$1 = std::vector of length 3, capacity 4 = {0x555555578020, 0x555555578070, 0x0}
(gdb) print children [2]
$2 = (WorkComponent *) 0x0
```

Thinking back to removingComposite() in demo.cpp we had the following lines

```
    WorkComponent* removed = project->remove(task);

    // since this is a short demo, instead of deleting removed, we add it
    back to the project

    cout << "[" << task->getName() << "]" has been moved from frontend to
    main project!\n";

    project->add(removed);
```

We then realised that WorkGroup::remove only searches the direct children of the group it is called on. Since “UI Design” is nested two levels deep, calling project->remove(task) correctly fails and returns nullptr. The calling code did not check before re-adding the result so a nullptr was pushed into the children vector of project. In a second GDB session, we break on remove and step past it with finish

```
(gdb) finish
Run till exit from #0  WorkGroup::remove (this=0x5555555788e0,
    child=0x555555578930) at WorkGroup.cpp:146
⚠ This WorkComponent could not be found in this WorkGroup!
0x00005555555558ab7 in removingComposite () at demo.cpp:191
191     WorkComponent* removed = project->remove(task);
Value returned is $8 = (WorkComponent *) 0x0
(gdb) quit
```

This confirmed that remove returned nullptr. The issue didn't happen then, it only happened during appendTo which appended without a null check, dereferencing the nullptr that was inserted. To fix this we did the following

Adjusted the demo.cpp to reference a direct child

```
WorkComponent* removed = frontend->remove(task);  
  
// since this is a short demo, instead of deleting removed, we add it  
back to the project  
  
cout << "[" << task->getName() << "]" has been moved from frontend to  
main project!\n";  
  
project->add(removed);
```

Adjusted WorkGroup.cpp to only add a child if it is not a nullptr

```
void WorkGroup::add(WorkComponent* child)  
{  
    if (!child)  
    {  
        cout << "⚠ Cannot add a null component!\n";  
        return;  
    }  
}
```

Together this ensures that the vector consistently works with only !nullptr values in terms of adding and removal. Now no nullptr ever enters the Composite structure.

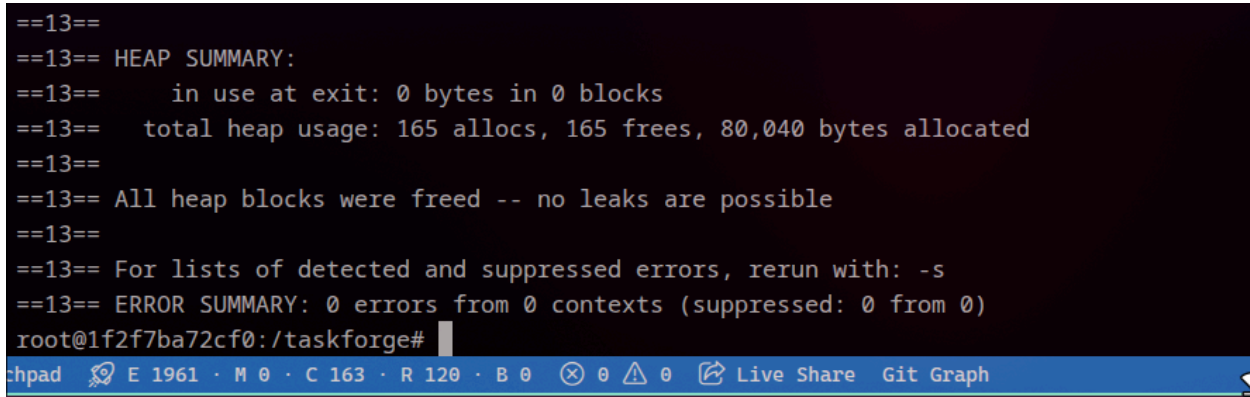
During this time we considered adjusting the remove() function to recursively search through the tree rather than only direct children, but determined it would not be suitable for the Composite pattern. add() and remove() work in tandem and the add() function does not recursively add a child, it adds a direct child to its caller, the remove() function being the opposite of add() only removes a direct child of the caller as well.

Here we have our Valgrind output for main.cpp, using docker to make use of Valgrind through the following command

```
> docker run --rm -it taskforge-image bash  
valgrind --leak-check=full --show-leak-kinds=all ./taskforge  
root@1f2f7ba72cf0:/taskforge# valgrind ./taskforge  
==13== Memcheck, a memory error detector  
==13== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.  
==13== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info  
==13== Command: ./taskforge  
==13==
```

After the program's execution, the Valgrind summary was as follows

```
==13==
==13== HEAP SUMMARY:
==13==    in use at exit: 0 bytes in 0 blocks
==13==  total heap usage: 165 allocs, 165 frees, 80,040 bytes allocated
==13==
==13== All heap blocks were freed -- no leaks are possible
==13==
==13== For lists of detected and suppressed errors, rerun with: -s
==13== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@1f2f7ba72cf0:/taskforge#
```



Task 6

We followed a workflow where roles rotated throughout the practical. All team members contributed to design discussion and implementation naturally divided based on strengths and availability.

During the design phase all members outlined the structure and pattern interactions, as well as agreed on the scenario. Senzo formulated the initial ideas whilst Amira validated and Jay consolidated.

Thereafter Jay implemented the core architecture and Senzo contributed to the initial main.cpp whilst debugging and investigating code which influenced the final scenario used. During this time, Amira took ownership of all diagrams which were reviewed by all team members after completion.

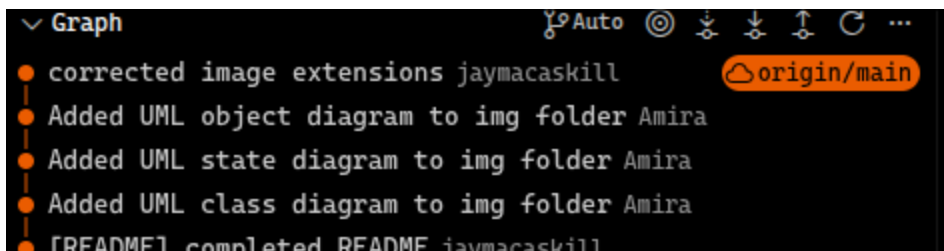
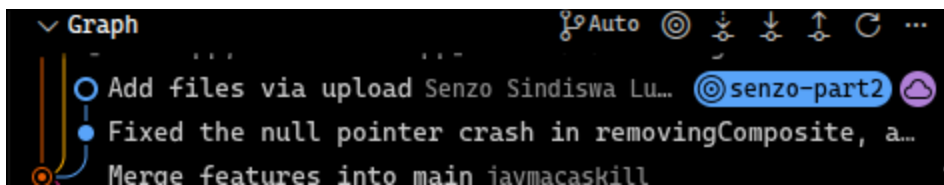
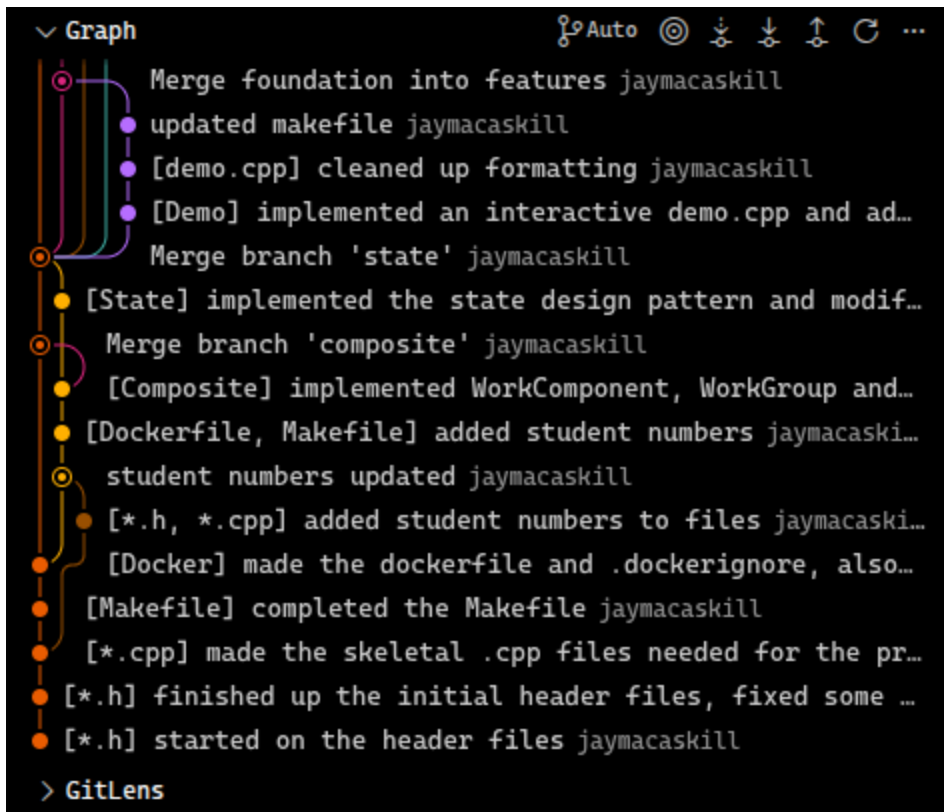
The final consolidation was done and commits show iterative development. Senzo identified a nullptr bug skillfully (2b43165) and the fix was integrated (a26f8e5).

Throughout, we followed the course's recommended pair programming model. When implementing new patterns, one team member wrote code, another reviewed the logic and ideas being written whilst another was testing.

Everyone also had their incredibly thorough role as tester by making use of the interactive demo.cpp.

Whilst the workload distribution skewed at times due to the compressed timeline, all teams contributed valuably and consistently. The team structure allowed for cross-review and debugging. In hindsight, more time could have been allocated to early diagram design to better balance written and coded contributions.

Below we see making use of Git and GitHub to frequently make meaningful commits to create a history of our work. Whilst laying foundations, making use of Git and GitHub made it less overwhelming and a lot clearer what expectations were and where to pick up from and easily “join in on the fun” at any point. This not only shows all three team members contributing, but also displays how GitHub allowed us to communicate continuously during development.



Task 7

- Identify where a GoF participant appears in the code
 - Table in task 1, and labelled in the class diagram
- Explain an ownership or lifetime decision
 - WorkGroup owns its children via a `vector<WorkComponent*>` and deletes them in its destructor, ensuring cascading cleanup. Decorators do own their wrapped components—`TaskDecorator::~~TaskDecorator()` calls `delete wrapped`—so when a `WorkGroup` deletes the outermost decorator, the entire decorator chain is cleaned up automatically. To avoid double deletion, `WorkGroup::decorate()` replaces the

original child with the decorator in the vector, transferring ownership to the decorator while the group only owns the outer layer

- Explain how traversal state is maintained and what happens after a runtime modification
 - Both iterators take a snapshot of the tree at construction time using `appendTo()`, so they are unaffected by later structural changes. If you move a task after building an iterator, that iterator still reflects the old structure, while a fresh iterator shows the updated hierarchy.
- Run or discuss GDB and Valgrind evidence
 - GDB revealed a segfault in our code caused by a null pointer inserted since we didn't check the return values. Valgrind confirmed all heap blocks were freed and there were zero memory leaks in the final product
- Explain a state transition or decorated behaviour
 - A task moves from `Todo` to `InProgress` when `next()` is called, and progresses to `Done` after four `execute()` calls. A `PriorityDecorator` automatically unblocks blocked tasks or boosts slow tasks, while a `LoggingDecorator` logs timestamps before and after execution
- Discuss the team's Git history
 - We primarily made use of a feature-branch workflow

Team Contribution Statement

Amira

- Created the UML Class Diagram
- Created the Object Diagram
- Created the State Diagram
- Created the three Activity Diagrams (Activity Diagram 1 - the hierarchy building and the traversal, Activity Diagram 2 - the task lifecycle, Activity Diagram 3 - snapshot iterator policy)

Senzo

- Set up SSH authentication for the repository.
- Created and pushed `senzo-part` (initial implementation) and `senzo-part2` (bug fix) branches for team review.
- Took part in the running of code average analysis and valgrind leak-check testing on the build.
- Implemented the core class logic of the system and wrote the initial `main.cpp`.
- Fixed several compile-time bugs, diagnosed a real segmentation fault using GDB.

Jay

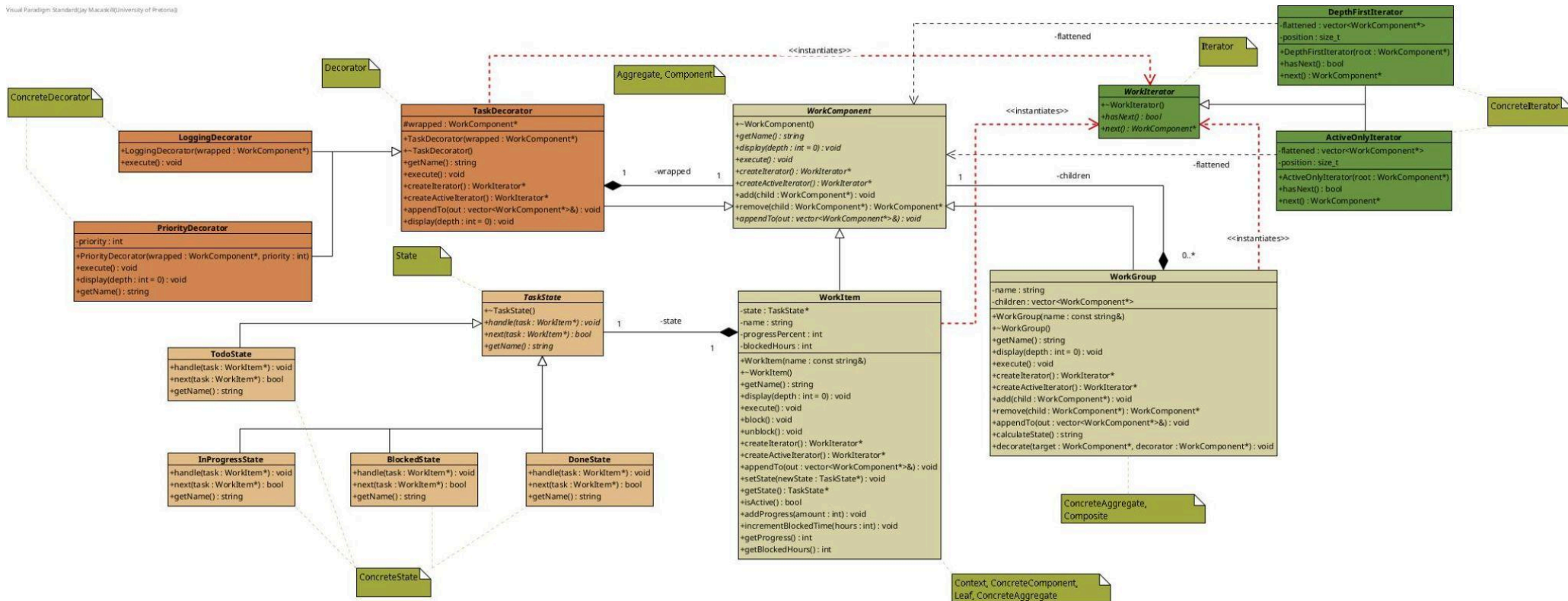
- Designed and implemented core architecture
- Initial implementations of patterns
- Skeletal files
- Integration of patterns into an interactive demo
- Review of static demo and expansion for coverage testing

- Wrote README
- Peer reviewed class, object and state diagrams
- Coordinated PDF compilation
- Created Git repository
- Reviewed team contributions

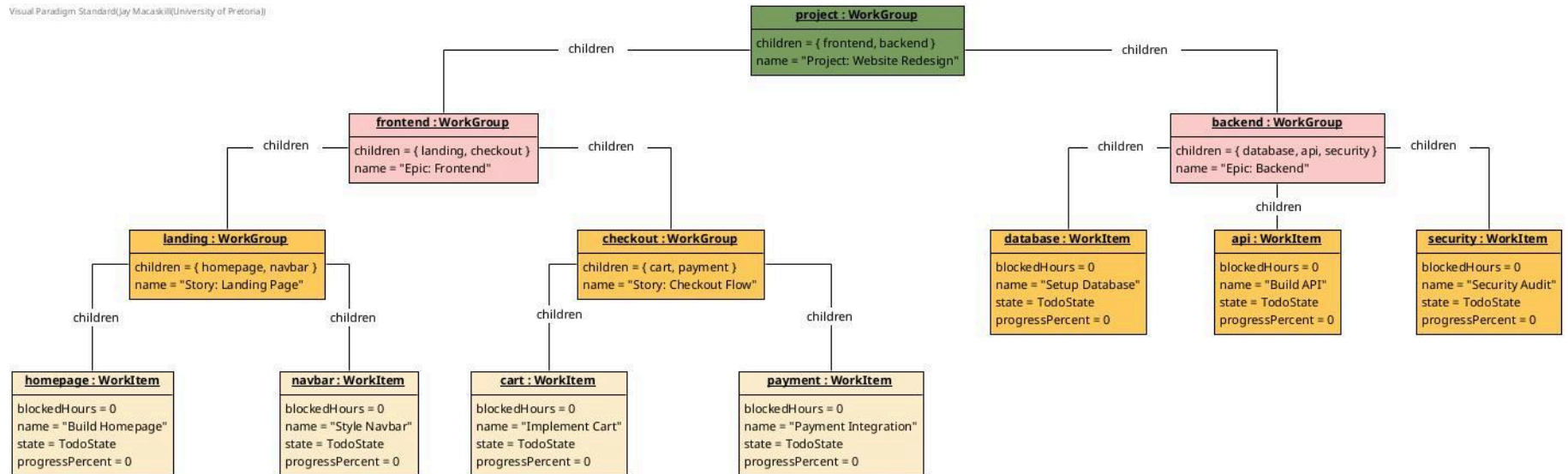
Enlarged Images

All images are also available on the github in the img folder: <https://github.com/jaymacaskill/code-library>

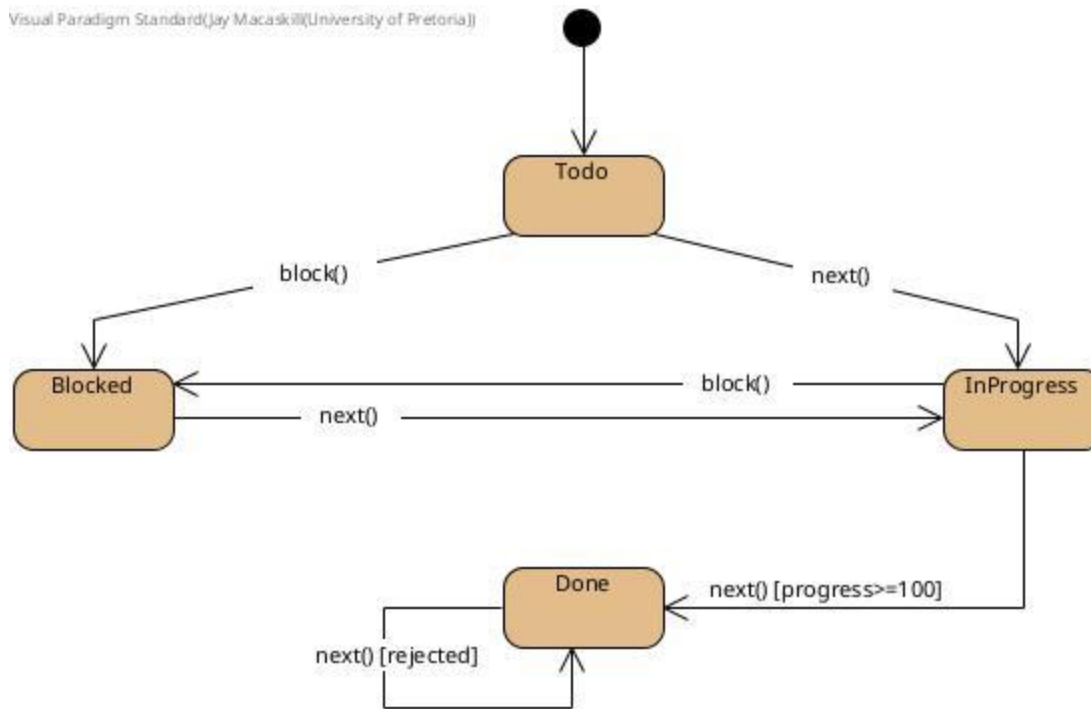
Visual Paradigm Standard(Jay Macaskill@University of Pretoria)



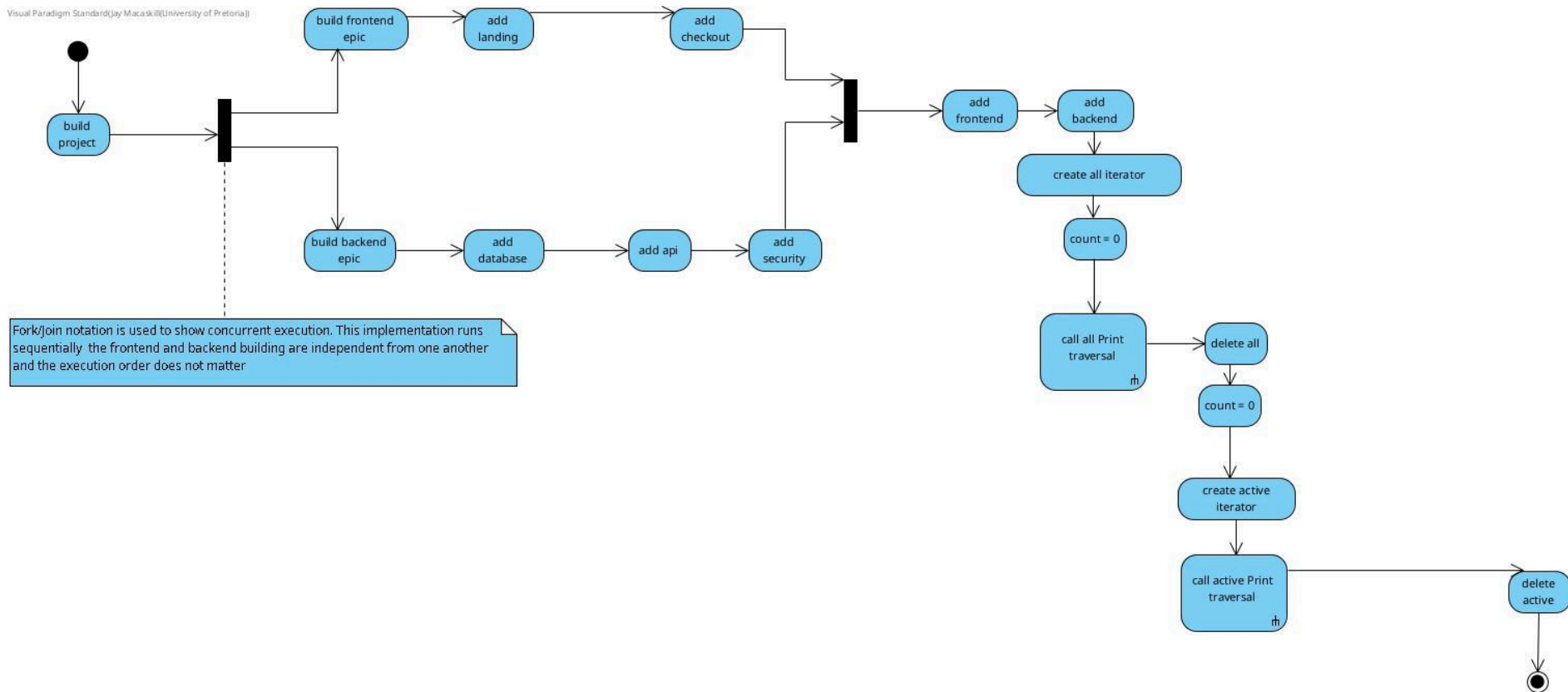
Task 1 UML Class Diagram



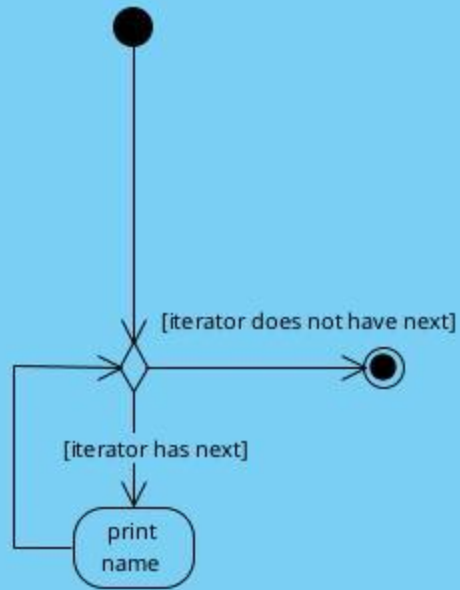
Task 4 UML Object Diagram



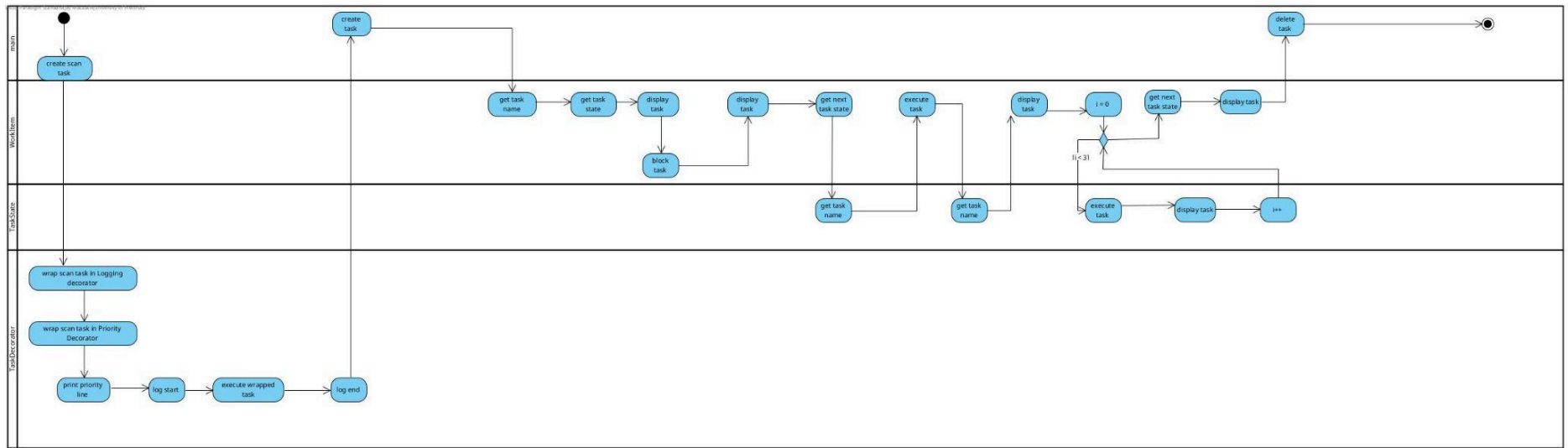
Task 4 UML State Diagram



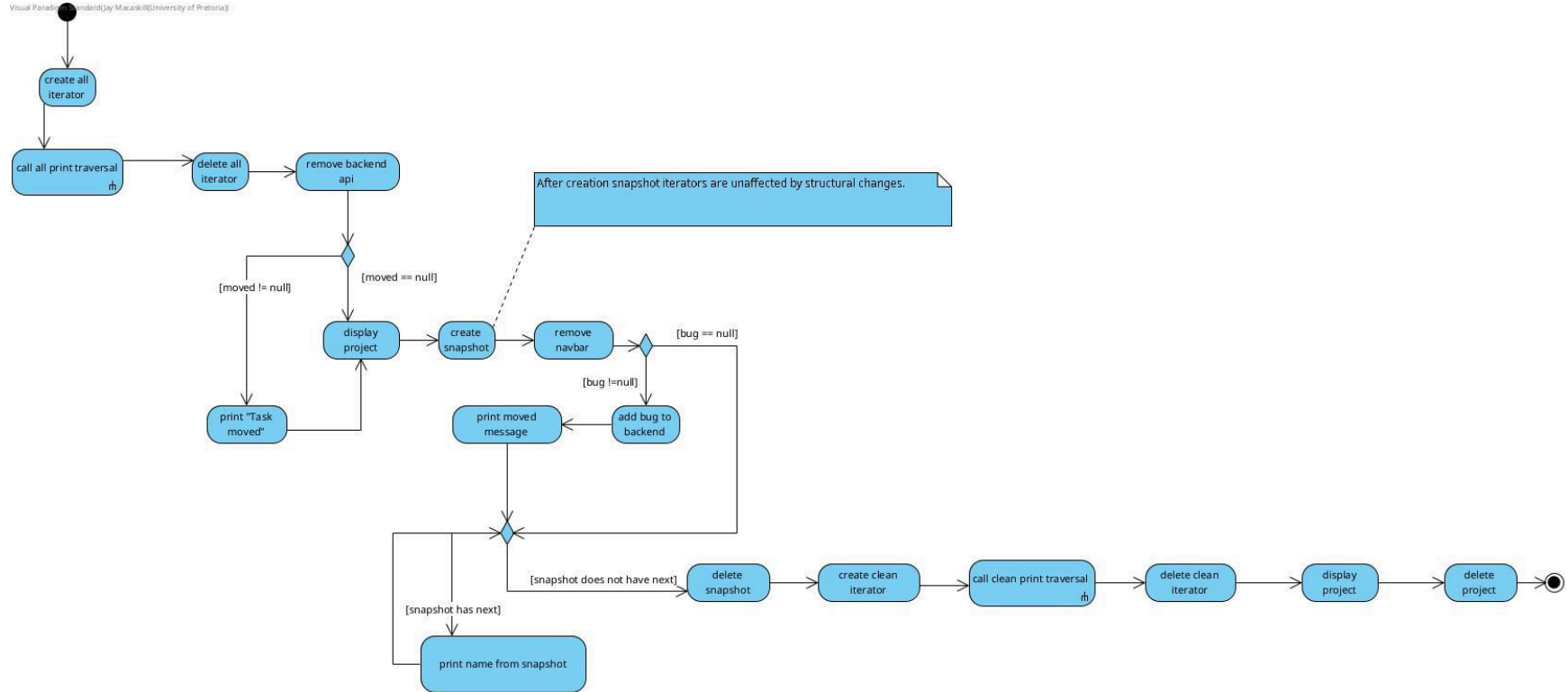
Print traversal



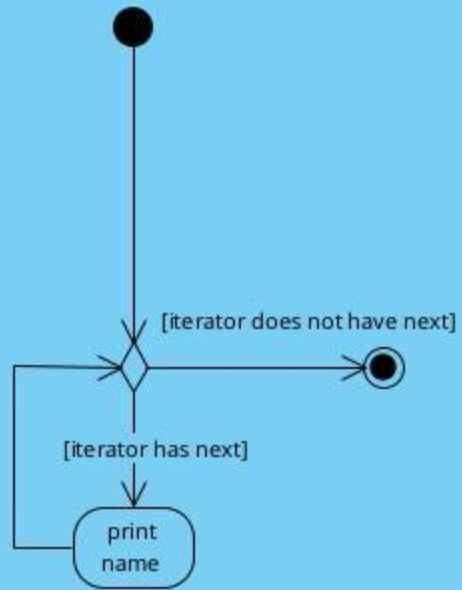
UML Activity Diagram (building & traversing)



UML Activity Diagram (Decorator & State lifecycle)



Print traversal



UML Activity Diagram (Iterator policy)